

Reviewer Pack — Minimal Symplectic Geometry Degree and Communication Complex...

Entry f125f7ceae97 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 16:29:16 UTC

Minimal Symplectic Geometry Degree and Communication Complexity Rank Variance

Entry ID: f125f7ceae97 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 16:29:16 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Communication Complexity

Statement:

For any given Boolean function f , the minimal symplectic geometry degree ($\text{msd}(f)$) of its associated vector space is linearly correlated with the variance in communication complexity rank ($\text{cvrank}(f)$), such that $\text{msd}(f) = \Theta(\text{cvrank}(f))$.

Rationale (proposer's reasoning):

Symplectic geometry has previously been applied to circuit complexity through the study of symplectic embeddings and their relation to approximation algorithms. This conjecture links this geometric structure to communication complexity, which is a measure of the minimum number of bits required for two parties to communicate information about a shared input. The variance in communication complexity rank might capture the diversity of interactions needed for information sharing.

Taxonomy category: symplectic_geometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d82064ae58c22d1e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if $\text{msd}(f)$ correlates with $\text{cvrank}(f)$ such that for at least 80% of seeds, the correlation coefficient is greater than or equal to 0.8 and the mean absolute difference between $\text{msd}(f)$ and $\text{cvrank}(f)$ is less than or equal to 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal symplectic geometry degree" AND "communication complexity rank variance" - "symplectic geometry" AND "communication complexity" AND "Boolean function" - "vector space associated with Boolean functions" AND "msd(f) = 0(cvrank(f))"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=25.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def symplectic_geometry_degree(f):
        n = len(f)
        count = 0
        for i in range(1 << (n-1)):
            if f[i] != f[~i]:
                count += 1
        return count

    def communication_complexity_rank(f):
        n = len(f)
        rank = 0
        for i in range(n):
            for j in range(i+1, n):
                if f[i] == f[j]:
                    continue
                rank += 1
        return rank

    instances_tested = 0
    msd_values = []
    cvrank_values = []

    for _ in range(30): # Ensure at least 30 instances per seed
```

```

n = random.choice([5, 10, 15, 20, 30, 40])
f = generate_boolean_function(n)
msd = symplectic_geometry_degree(f)
cvrank = communication_complexity_rank(f)

if msd is None or cvrank is None:
    continue

instances_tested += 1
msd_values.append(msd)
cvrank_values.append(cvrank)

if instances_tested == 0:
    return {
        "metric_name": "msd vs cvrank",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

mean_ms = sum(msd_values) / instances_tested
mean_cvrank = sum(cvrank_values) / instances_tested

correlation_coefficient = sum((msd - mean_ms) * (cvrank - mean_cvrank) for msd, cvrank in zip(

return {
    "metric_name": "msd vs cvrank",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": max(len(f) for f in msd_values + cvrank_values),
    "conjecture_holds": abs(correlation_coefficient) >= 0.8 and mean(abs(msd - cvrank)) <= 3,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] + list(range(31))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
        std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
        support_fraction = 1.0
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        counterexample = "mapping_undefined"
        mean_metric_value = None
        std_metric_value = None
        support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

```

```
print(f"RESULT: {'SUPPORTED' if all(r['conjecture_holds'] for r in results) else 'FALSIFIED'}
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c3de944e.py", line 89, in <module>

result = run_trial(seed)

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c3de944e.py", line 49, in run_trial

msd = symplectic_geometry_degree(f)

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c3de944e.py", line 28, in symplectic_ge

if f[i] != f[~i]:

~^^^

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's correlation coefficient and mean absolute difference as required by the support condition. | next: Investigate the cause of the crash in the test code to ensure it can run successfully and provide the necessary data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12466
2	preregistration	ollama_remote	glm4:latest	0	0	9610
3	novelty	ollama_remote	glm4:latest	0	0	8508
4	novelty	ollama_remote	glm4:latest	0	0	8742
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14611
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10270
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10024
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10408
9	judge	ollama_remote	glm4:latest	0	0	29857

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 114495 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/f125f7ceae97.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f125f7ceae97.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f125f7ceae97.tar.gz (if generated)